Experiment No. 11 (VA 11)

Title

Dynamic Class Scheduler & Countdown using JavaScript Timers (setTimeout(), setInterval()) and ES6 Modules (import/export)

Aim

To use JavaScript timers (setTimeout(), setInterval()) and ES6 modules (import/export) to build a dynamic countdown / class scheduler with real-time updates and a responsive layout.

Software / Tools Required

- Visual Studio Code (VS Code)
- Google Chrome

Theory

This experiment explores time-based programming and modular code organisation in JavaScript. The Class Scheduler & Countdown application shows the current time, tracks the upcoming class from a routine, and runs a controllable countdown that updates the interface in real time.

- **setInterval():** Repeatedly executes a function at a fixed delay. It refreshes the live clock every second and decrements the countdown by one second on each tick; clearInterval() stops it when paused or reset.
- **setTimeout():** Executes a function once after a specified delay. It is used to trigger the class-start notification and to clear temporary status messages after a short time.
- **ES6 Modules:** Code is split into separate files using export and import (e.g. routine data, timer logic, UI updates), improving reusability and maintainability. Loaded with <script type="module">.
- **Class Scheduling Logic:** The class routine (subject, venue and duration) is stored as an array of objects. The scheduler picks the next class, loads its duration into the countdown, and moves on when the timer reaches zero.
- **Timer Controls:** Start, Pause, Reset and a custom "Enter seconds" input with a Set button let the user control the countdown through DOM event listeners.
- **Real-Time DOM Updates:** The clock, countdown, upcoming-class heading and status message (e.g. "Started: Web Technologies!") are updated dynamically via textContent, without reloading the page.
- **Responsive Layout:** Flexbox/CSS with relative units and a centred card layout keeps the interface readable on different screen sizes.

Output:

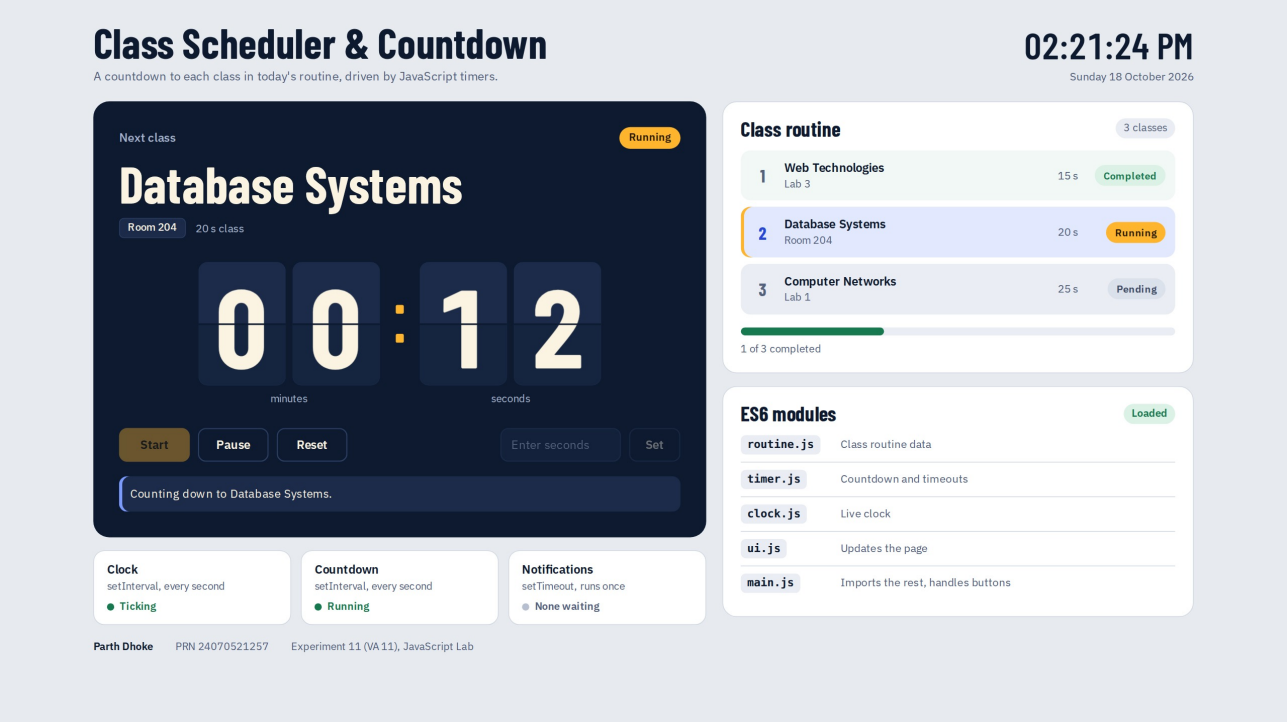


Fig. 1: Countdown running for the next class

Result / Conclusion

In conclusion, this experiment demonstrated the use of **JavaScript timers and ES6 modules** by developing a functional Class Scheduler & Countdown application. setInterval() provided real-time clock and countdown updates, setTimeout() handled timed notifications, and import/export kept the code modular. The Start, Pause, Reset and Set controls worked correctly and the responsive interface updated dynamically. Thus, the experiment demonstrated effective use of **timers, modules, and DOM manipulation** in an interactive web application.